SOTERIX MEDICAL MXN-33 LSL INTEGRATION MANUAL

September 2023

Version 1.2

BETTER. SMARTER.
Soterix medical

Contents

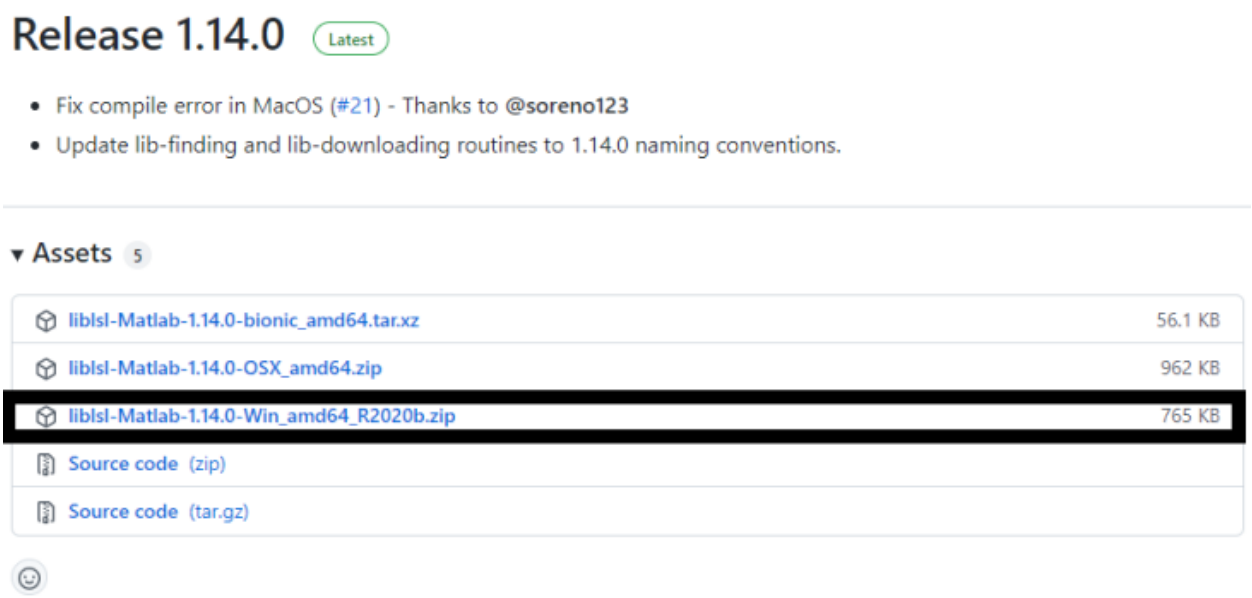
Downloading MATLAB code

The Soterix Medical HD-SC application is provided on a Laptop with the MXN-33 devices. For LSL integration, users can download the library for windows operating systems as per the instructions below.

The detailed steps along with the screenshots are as follows:

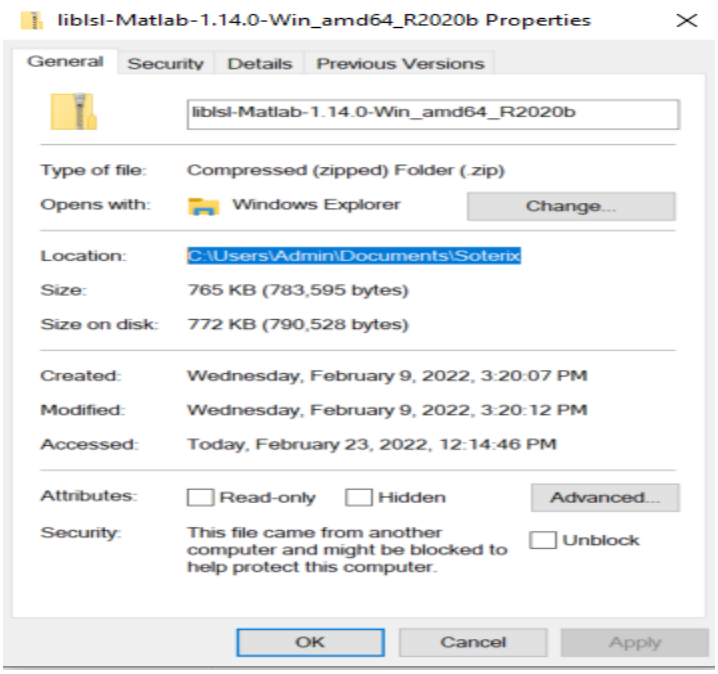
Github code

- Download the MATLAB LSL library from here- https://github.com/labstreaminglayer/liblsl-Matlab/releases



To proceed, locate the downloaded LSL library and the HD-SC file that is already on the computer, and place them in the same file location. Copy the location by right clicking the file and then going down to “properties”.

Example:



The HD-SC file will look like:

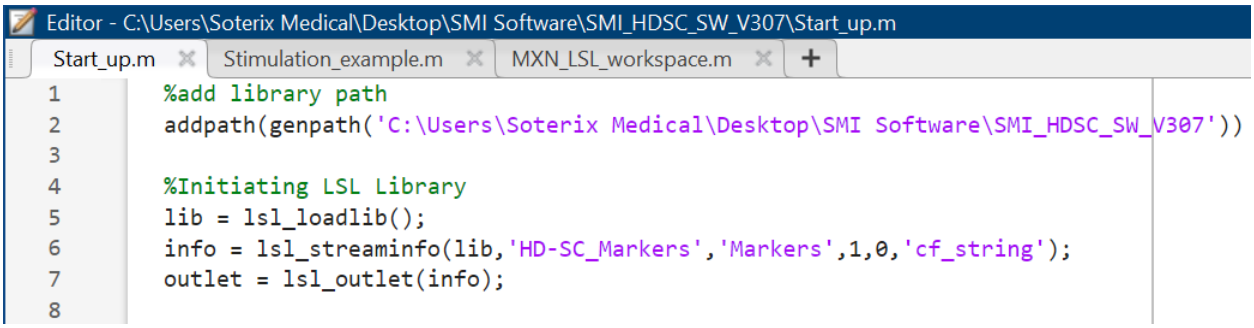
Name	Date modified	Type
HD-SC.exe.config	2/9/2022 3:39 PM	CONFIG File
HD-SC	2/9/2022 3:39 PM	Application
waveforms_files - Copy	2/16/2022 2:08 PM	File folder
waveforms_files	4/26/2022 11:36 AM	File folder
SystemFiles	2/9/2022 4:42 PM	File folder
data	2/9/2022 4:42 PM	File folder

Add Path

There are three MATLAB script files provided by Soterix Medical and they are Start_up.m, Stimulation_example.m, and MXN_LSL_Workspace.m.

The location to implement the library path is in Start_up.m. Paste the location of the library in “addpath(genpath(“Paste location”)).

Example:



After the code is run, MATLAB will be connected to the GUI.

Operating the GUI through MATLAB

The following section discusses how to adjust almost all functions in the GUI through MATLAB. Exceptions currently are switching between Basic and Arbitrary modes, Polarity, Ramp Occurrence, changing MXN Version, Amplitude Modulation, and Compact view. Most operations in MATLAB have a similar process, meaning once the user understands how to adjust the individual intensity, they will know how to adjust most operations.

It is important for the user to know the “Action codes”. These Action codes can be referenced in the MXN_LSL_Workspace.m file, but for clarity’s sake see below. Whenever there is a function with the right sided number (starting from 0) following ‘Action’, the definition will tell you what the function is trying to do.

The following are Action codes:

1. Add Channel = 0
2. Add Channel from File = 1
3. Delete Channel = 2
4. Load Device = 3
5. Start Stimulation = 4
6. Stop Stimulation = 5
7. Update Settings = 7
8. Change Impedance = 8

Basic/Arbitrary Mode

Basic and Arbitrary mode cannot be controlled from MATLAB. The mode required must be clicked on prior to running the MATLAB code.

Individual Intensity

The Action code for individual intensity is “updating the settings”. As shown above that Action is referenced to the number 7 therefore 7 follows the Action within the code. ‘Intensity’ is what is being updated. The number following ‘intensity’ is the intensity desired by the user in mA. The following JSONIntensity line converts the MATLAB code into the library, which will tell the GUI what to do.

```
%Settings: Intensity
pIntensity = struct('Action',7,'Intensity',2);
JSONIntensity = jsonencode(pIntensity);

outlet.push_sample({JSONIntensity});
pause(2)
```

Duration, Delay, Rampup, and Waveform

Similar functions will apply to duration, delay, ramp up/ramp down, and waveform.

Example:

```
%Settings: Duration
pDuration = struct('Action',7,'Duration',20);
JSONDuration = jsonencode(pDuration);

outlet.push_sample({JSONDuration});
pause(2)


%Settings: Delay
pDelay = struct('Action',7,'Delay',50);
JSONDelay = jsonencode(pDelay);

outlet.push_sample({JSONDelay});
pause(2)


%Settings: RampUp
pRampUp = struct('Action',7,'RampUp',3);
JSONRampUp = jsonencode(pRampUp);

outlet.push_sample({JSONRampUp});
pause(2)
```

Note: ‘RampDown’ will not work in MATLAB. Whichever number assigned for ramp up will be synchronized into ramp down.

```
%Settings: Waveform Change tACS
ptACS = struct('Action',7,'WaveformType','tACS');
JSONtACS = jsonencode(ptACS);

outlet.push_sample({JSONtACS});
pause(2)
```

This above function can change the waveform to tACS, tRNS, and tDCS, but not to amplitude modulation.

The next four operations are intuitive. They will either have a different Action associated to them or there will be multiple operations attached to a single function.

SHAM and Channel Selection

```
%Settings: Waveform Change tACS
ptACS =
struct('Action',7,'WaveformType','tACS','SHAM','FALSE','ChannelSelection','MULTIPLE')
;
JSONtACS = jsonencode(ptACS);

outlet.push_sample({JSONtACS});
pause(2)
```

This function will change the waveform to tACS, turn SHAM off, and change the channel selection to multiple.

If the desired output is for SHAM to be on and the channel selection to be single, the function will look like:

```
%Settings: Waveform Change tACS
ptACS =
struct('Action',7,'WaveformType','tACS','SHAM','true','ChannelSelection','SINGLE');
JSONtACS = jsonencode(ptACS);

outlet.push_sample({JSONtACS});
pause(2)
```

Add Channel

The Action for adding a channel is 0.

```
% Add Channel
pChannel = struct('Action',0,'ChannelNumber',25);
JSONaddChannel = jsonencode(pChannel);

outlet.push_sample({JSONaddChannel});
pause(2)
```

In the example above, Channel 25 is added.

Assuming the channel selection is multiple, as many as 32 channels can be added.

```
% Add Channel
pChannel = struct('Action',0,'ChannelNumber',25);
JSONaddChannel = jsonencode(pChannel);

outlet.push_sample({JSONaddChannel});
pause(2)

pChannel18 = struct('Action',0,'ChannelNumber',18);
JSONaddChannel18 = jsonencode(pChannel18);

outlet.push_sample({JSONaddChannel18});
pause(2)

pChannel14 = struct('Action',0,'ChannelNumber',14);
JSONaddChannel14 = jsonencode(pChannel14);

outlet.push_sample({JSONaddChannel14});
pause(2)


pChannel31 = struct('Action',0,'ChannelNumber',31);
JSONaddChannel31 = jsonencode(pChannel31);

outlet.push_sample({JSONaddChannel31});
pause(2)
```

Impedance

This function is structured so that it is disabling the impedance. This means that if the desired output is for the impedance to be turned off, then the value of the code will be 'true'. In contrast to that, the value of the code will be false for the impedance to be on.

```
% Disable Impedance
pDisableImpedance = struct('Action',8, 'value', 'TRUE');
JSONDisableImpedance = jsonencode(pDisableImpedance);

outlet.push_sample({JSONDisableImpedance});
pause(2)
```

Impedance is off.

```
% Enable Impedance
pDisableImpedance = struct('Action',8, 'value', 'FALSE');
JSONDisableImpedance = jsonencode(pDisableImpedance);

outlet.push_sample({JSONDisableImpedance});
pause(2)
```

Impedance is on.

To disable/enable the impedance, the function pDisabeImpedance needs to be repeated, regardless of the output desired.

Example:

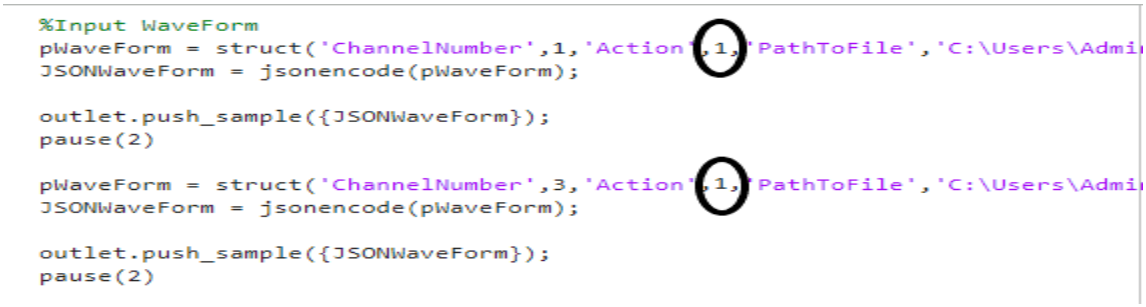
```
% Disable Impedance
pDisableImpedance = struct('Action',8, 'value', 'TRUE');
JSONDisableImpedance = jsonencode(pDisableImpedance);

outlet.push_sample({JSONDisableImpedance});
pause(2)

pDisableImpedance = struct('Action',8, 'value', 'TRUE');
JSONDisableImpedance = jsonencode(pDisableImpedance);

outlet.push_sample({JSONDisableImpedance});
pause(2)
```

Waveform Graph

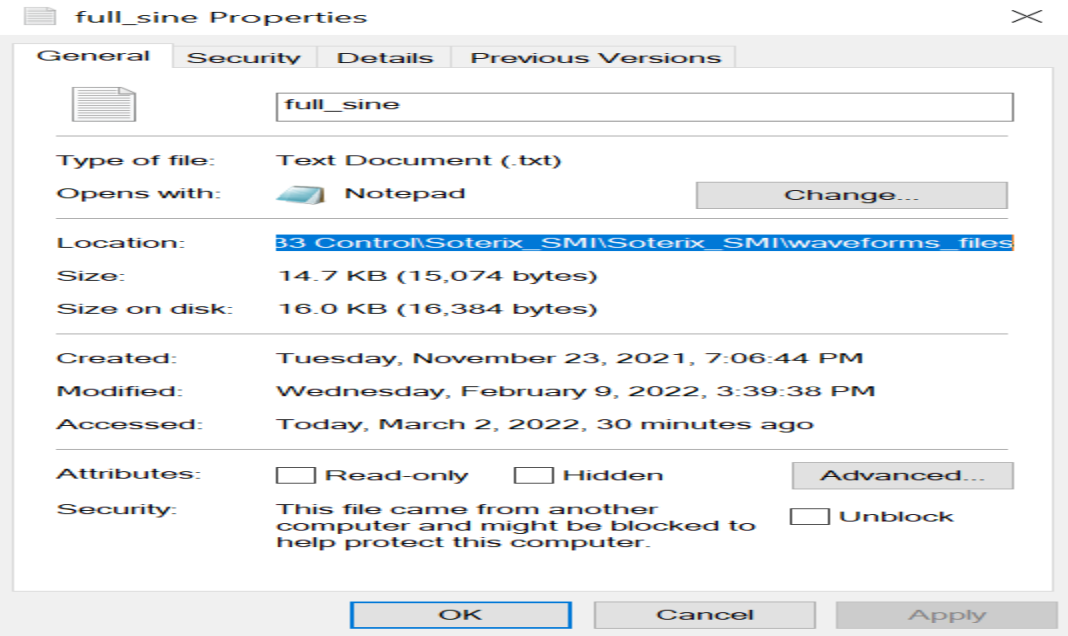
To implement a waveform graph, a channel and a specified graph needs to be within the function.

```
%Input WaveForm
pWaveForm =
struct('ChannelNumber',1,'Action',1,'PathToFile','C:\Users\Admin\Documents\Soterix\MX
N-33 Control\Soterix_SMI\Soterix_SMI\waveforms_files/full_sine.txt');
JSONWaveForm = jsonencode(pWaveForm);

outlet.push_sample({JSONWaveForm});
pause(2)
```

Action 1 or Action 0 can be used to add a channel. However, if multiple channels and waveforms are being added the Action code will need to be consistent.



To specify the graph associated to that channel, the user must first copy its location – use page 5 as a guide – then add the name of that file as a text file (.txt) after a forward slash.



An example of implementing the full sine waveform.

It is futile to add both a channel (as was done on page 9) and to add a channel through the waveform code since only one will show on the GUI. Therefore, if multiple graphs are required, the user will need to add a waveform code for each channel.

```
%Input Waveform
pWaveForm = struct('ChannelNumber',18,'Action',1,'PathToFile','C:/Users/Admin...');
JSONWaveForm = jsonencode(pWaveForm);

outlet.push_sample({JSONWaveForm});
pause(2)

pWaveForm = struct('ChannelNumber',14,'Action',1,'PathToFile','C:/Users/Admin...');
JSONWaveForm = jsonencode(pWaveForm);

outlet.push_sample({JSONWaveForm});
pause(2)
```

Frequency

Once a channel is added only then can the frequency be adjusted. The Action code is 7.

```
%Settings: Frequency
pFrequency = struct('Action',7,'ChannelNumber',18,'Frequency',1.8);
JSONFrequency = jsonencode(pFrequency);


outlet.push_sample({JSONFrequency});
pause(2)
```

Load, Start, and End

The load must be before the start function; page 7 shows which Actions are needed for load, start, and end.

```
%Load Device

pLoad = struct('Action',3, 'value', 'TRUE');
JSONLoad = jsonencode(pLoad);

outlet.push_sample({JSONLoad});
pause(2)

%Starting Stimulation

pstartStimulation = struct('Action',4,'value', 'TRUE');
JSONstartStimulation = jsonencode(pstartStimulation);

outlet.push_sample({JSONstartStimulation});
pause(2)


%Stopping Stimulation

pstopStimulation = struct('Action',5,'value', 'TRUE');
JSONstopStimulation = jsonencode(pstopStimulation);

outlet.push_sample({JSONstopStimulation});
pause(2)
```

The user can delay the stopping stimulation by editing the pause() part of the starting stimulation function. The integer implemented will represent the time delay in seconds.

For example:

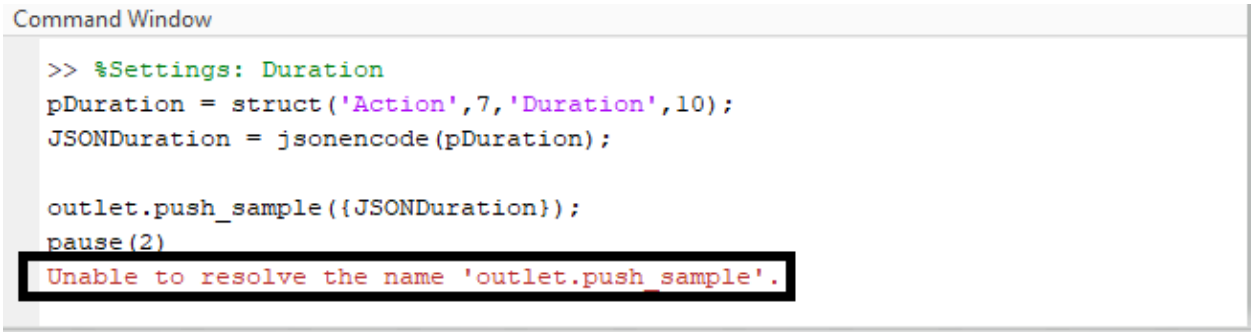
```
%Load Device

pLoad = struct('Action',3, 'value', 'TRUE');
JSONLoad = jsonencode(pLoad);

outlet.push_sample({JSONLoad});
pause(2)

%Starting Stimulation

pstartStimulation = struct('Action',4,'value', 'TRUE');
JSONstartStimulation = jsonencode(pstartStimulation);

outlet.push_sample({JSONstartStimulation});
pause(9)

%Stopping Stimulation

pstopStimulation = struct('Action',5,' value', 'TRUE');
JSONstopStimulation = jsonencode(pstopStimulation);

outlet.push_sample({JSONstopStimulation});
pause(2)
```

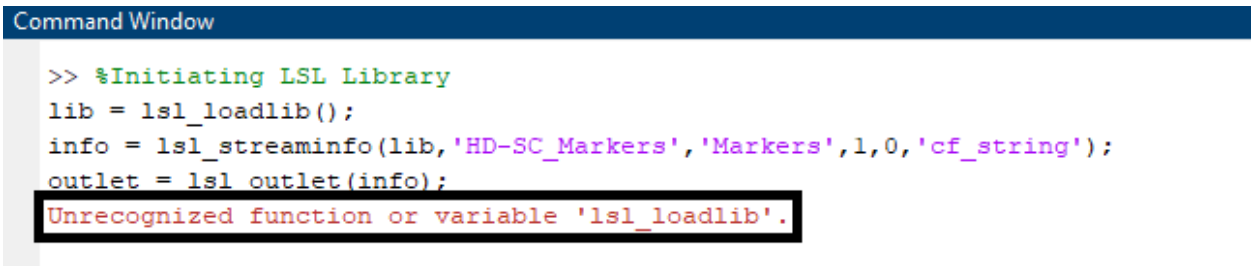
Possible Errors

The following section is dedicated to common complications users can run into.

Start-up of MATLAB and HD-SC

If the user cannot control the GUI through MATLAB or receives the following errors (see screenshot below). It means that the order in which one must start the MATLAB code and the GUI was not run properly.





The order to successfully control the HD-SC software using MATLAB is as follows:

Open the HD-SC software, run the Start_up.m code on MATLAB, log in to HD-SC, and then press 'connect'.

The "Initiating LSL Library "code cannot be run on the same page as the script.

The below script for example will not run.

```
%Initiating LSL Library
lib = lsl_loadlib();
info = lsl_streaminfo(lib,'HD-SC_Markers','Markers',1,0,'cf_string');
outlet = lsl_outlet(info);

%Settings: Duration
pDuration = struct('Action',7,'Duration',16);
JSONDuration = jsonencode(pDuration);

outlet.push_sample({JSONDuration});
pause(2)

% Add Channel
pChannel = struct('Action',0,'ChannelNumber',25);
JSONaddChannel = jsonencode(pChannel);

outlet.push_sample({JSONaddChannel});
pause(2)

pChannel18 = struct('Action',0,'ChannelNumber',18);
JSONaddChannel18 = jsonencode(pChannel18);

outlet.push_sample({JSONaddChannel18});
pause(2)
```

Minimum Duration

The duration has a minimum value based on two things: what the ramp is and whether SHAM is turned on or off. If SHAM is off, the duration is the ramp multiplied by 3 + 1. If SHAM is on, the duration is in intervals of four - based on the ramps value. Below are the tables of the two conditions and what the minimum duration would be if ramp is from 1 to 10.

SHAM OFF

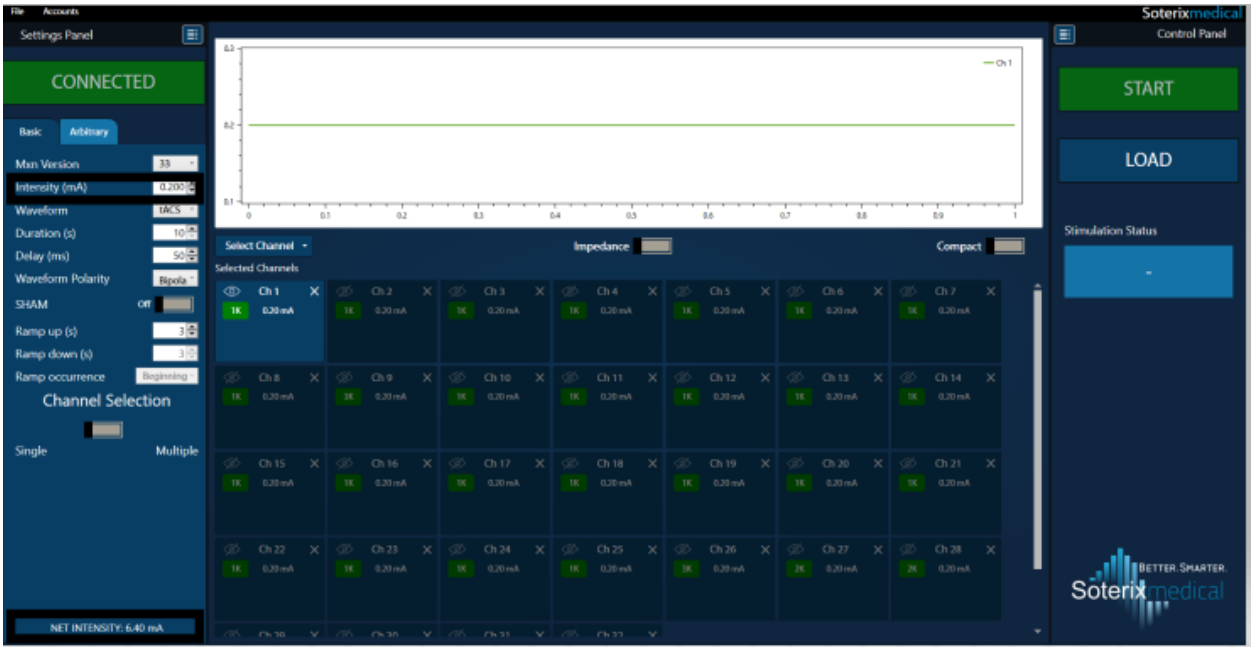
Ramp Up/Down	Minimum Duration
1	4
2	7
3	10
4	13
5	16
6	19
7	22
8	25
9	28
10	31

SHAM ON

Ramp Up/Down	Minimum Duration
1	5
2	9
3	13
4	17
5	21
6	25
7	29
8	33
9	37
10	41

Single Channel Selection

To use the Single channel selection, the first channel must be added and the Net Intensity (located on bottom left of screen) needs to be under 10 mA.



The Net Intensity is calculated by multiplying the Individual Intensity by 32 channels. In this example it is 0.2 mA * 32 = 6.4 mA.

Specifically, when using the single channel selection there needs to be two pstartSimulation functions.

Example:

```
%Starting Stimulation

pstartStimulation = struct('Action',4,'value', 'TRUE');
JSONstartStimulation = jsonencode(pstartStimulation);

outlet.push_sample({JSONstartStimulation});
pause(2)

pstartStimulation = struct('Action',4,'value', 'TRUE');
JSONstartStimulation = jsonencode(pstartStimulation);

outlet.push_sample({JSONstartStimulation});
pause(2)
```

Channel Selection Is Not Showing & Impedance Toggle Does Not Work

Both can be resolved by adjusting the resolution on the screen.

If the resolution is good and the impedance toggle still is not working – press connect on the GUI (top left of the screen) while the computer is plugged into the MXN-33.

Frequency

The function pFrequency will only work if a channel is added prior. It does not matter whether pChannel or pWaveForm are used. If pWaveForm is used, then that mode that will be arbitrary. However, if pChannel is used then the waveform must not be tDCS, otherwise there will be no frequency.

```
%Settings: Add Channel
pChannel = struct('Action',0,'ChannelNumber',3);
JSONaddChannel = jsonencode(pChannel);

outlet.push_sample({JSONaddChannel});
pause(2)

%Settings: Frequency

pFrequency = struct('Action',7,'ChannelNumber',3,'Frequency',1.8);
JSONFrequency = jsonencode(pFrequency);

outlet.push_sample({JSONFrequency});
pause(2)
```

OR

```
%Input Waveform
pWaveForm = struct('ChannelNumber',3,'Action',1,'PathToFile','C:/Users/Admin...');
JSONWaveForm = jsonencode(pWaveForm);

outlet.push_sample({JSONWaveForm});
pause(2)

%Settings: Frequency

pFrequency = struct('Action',7,'ChannelNumber',3,'Frequency',1.8);
JSONFrequency = jsonencode(pFrequency);

outlet.push_sample({JSONFrequency});
pause(2)
```

Amplitude Modulation Frequency

When trying to adjust the frequency when the waveform is at “Amplitude Modulation” there cannot be a waveform graph assigned to a channel in the script. Therefore, the above screenshot will not work In AM mode. However, the below code will.

```
%Settings: Duration
pDuration = struct('Action',7,'Duration',16);
JSONDuration = jsonencode(pDuration);

outlet.push_sample({JSONDuration});
pause(2)

%Settings: Add Channel
pChannel = struct('Action',0,'ChannelNumber',3);
JSONaddChannel = jsonencode(pChannel);

outlet.push_sample({JSONaddChannel});
pause(2)

%Settings: Frequency
pFrequency = struct('Action',7,'ChannelNumber',3,'Frequency',1.8);
JSONFrequency = jsonencode(pFrequency);


outlet.push_sample({JSONFrequency});
pause(2)
```

Since this waveform cannot be adjusted through MATLAB, the user must manually adjust the waveform at AM before running the code.

Order of Operations

The order of operations that are needed in all parts of this document are summarized below:

Running MATLAB and HD-SC:

(1) Open HD-SC
(2) Run Start_up.m on MATLAB
(3) Log in to HD-SC (username and password)
(4) Press Connect

Frequency: Frequency must be added only after adding a channel or a waveform graph.

Load, start, end: To run the code it must be loaded, then started, and then ended (optional). It cannot be loaded till the laptop is connected to the MXN-33, and the green bar “connect” is pressed.

Code is Not Starting

There are a few solutions to this issue, as always, the GUI can be rebooted and that will solve the problem occasionally. Other common errors are:

1. The Channel Selection is on single, and the above parameters were not followed (pages 18, 19 and 20).
2. There is only one pDisableImpedance code. (Page 12)
3. There is only one load function. As of now there needs to be two pLoad functions for the user to have loading flexibility.
4. When using arbitrary mode, the waveform type cannot be tDCS.
5. Using double quotes in the code instead of a single quote (‘).
6. Both the LSL library and the HD-SC file must be extracted before the code can run. Right click the file and click “extract all”.